EE5203 Lab 4 Guide

GPIO with the HAL: same pins, new interface - Basri Erdogan

Mission: Rebuild your Lab 3 LED control with HAL functions, let B1 control the LED on PA5 by polling, and show in the register view that HAL wrote the same bits you wrote by hand in Lab 3.

This lab uses Lecture 5 (HAL GPIO, inputs, pull resistors, polling). You saw the first two steps as live demos in the lecture.

Prepare before class

- ☐ Bring your board and a USB **data** cable.
- ☐ Do the **Lab 4 pre-lab quiz** on CATS.
- ☐ Re-read the theory slides up to "Polling B1 to control LD2".
- ☐ Know why B1 released reads `GPIO_PIN_SET` (external pull-up R30, active-low).

Learning objectives

By checkoff time, you can:

1. configure PA5 (output) and PC13 (input, no pull) in CubeMX and find them in the generated `MX_GPIO_Init()`;
2. drive and read pins with `HAL_GPIO_WritePin`, `HAL_GPIO_TogglePin` and `HAL_GPIO_ReadPin`;
3. check a HAL configuration in the register view, down to the field;
4. detect a button **press** (the change from released to pressed), not only its level.

Hardware and files

Item	Value
Board	Nucleo-F401RE: LED on PA5, B1 on PC13, no wiring
Tools	STM32CubeMX (new project) + VS Code
Project name	`EE5203_L04_<studentnumber>`
File you edit	`main.c`, only inside the `USER CODE` regions
Register fields	`GPIOA MODER5`; `GPIOC MODER13`, `PUPDR13`, `IDR13`

ℹ Using your own laptop? (click)

Follow the steps as written, with these changes. Set up your laptop once with the Home Setup Guide.

- **Folder:** `C:\STM32\EE5203` (Mac: `~/STM32/EE5203`), not `C:\Work`.
- **Step 4:** skip it. Open the folder, choose **Debug**, then ST icon → **Set Up STM32Cube project(s)**.
- **Build / debug: F7** (Mac: **fn + F7**), then Run and Debug → **STM32Cube: STM32 Launch ST-Link GDB Server**.
- **Registers:** use **STM32CUBE REGISTERS TREE** wherever this guide says XPERIPHERALS.
- **End of session:** no *Save to H*. Your project stays on your laptop.

Checkpoint A - New HAL project and register proof

Start of the session: if `C:\Work\<studentnumber>` does not contain your Lab 3 project, copy **all** your project folders from `H:\EE_Labs` into it first.

1. **STM32CubeMX:** Board Selector → **NUCLEO-F401RE** → accept the default initialization. Then set up **PA5** yourself:
   - Click **PA5** → **Reset_State**. The pin is now unused (grey).
   - Click **PA5** again → `GPIO_Output` (green).
   - In System Core → GPIO, PA5: **User Label** empty, **Low** output level, **Output Push Pull**, **No pull-up and no pull-down**, **Low** speed.

   With no label, `main.c` names the pin `GPIOA, GPIO_PIN_5`.
2. Click **PC13** and choose `GPIO_Input` (the default is `GPIO_EXTI13`). In System Core → GPIO, PC13: **No pull-up and no pull-down**.
3. Project Manager: name `EE5203_L04_<studentnumber>`, location `C:\Work\<studentnumber>`, Toolchain/IDE = **CMake** → Generate Code.
4. Copy `C:\Users\Public\EE5203_template\.vscode` into the project folder.
5. In `main.c`, check that `main()` calls `MX_GPIO_Init()` once before the loop. Do not add a second call.
6. Inside `USER CODE BEGIN 3`:

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500);
```

7. Build (**Ctrl+Shift+B**), start the debugger (**F5**), Continue: the LED on PA5 blinks.
8. Stop at a breakpoint after `MX_GPIO_Init()` and open **XPERIPHERALS**:

Register → field	Expected	Meaning
GPIOA → MODER → `MODER5`	`01`	output (as in your Lab 3 table)
GPIOC → MODER → `MODER13`	`00`	input
GPIOC → PUPDR → `PUPDR13`	`00`	no internal pull

How to read the register fields (click)

In XPERIPHERALS each pin has a named field: `MODER5`, `PUPDR13`, `IDR13`…

Field	Values
`MODERy` (2 bits)	`00` input · `01` output · `10` alternate function · `11` analog
`PUPDRy` (2 bits)	`00` no pull · `01` pull-up · `10` pull-down · `11` reserved
`IDRy` (1 bit, read-only)	the level on the pin now: `1` high, `0` low
`ODRy` (1 bit)	the value an output pin drives

Pause the program first: the register view only updates while the program is halted. Full register views: L05 theory slides 30–31.

Show: the LED on PA5 blinking and the three fields in XPERIPHERALS.

Checkpoint B - B1 stops the blink

Replace the two lines in `USER CODE BEGIN 3`:

```
GPIO_PinState button = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);

if (button == GPIO_PIN_SET) {          /* released: blink */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(500);
} else {                               /* pressed: LED off */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
}
```

Before you run it, predict the LED on PA5 for "released" and "pressed". Then put a breakpoint on the `if` line. Released: `button` = `GPIO_PIN_SET` and `IDR13` = 1. Hold B1 and press Continue: `GPIO_PIN_RESET` and `IDR13` = 0.

Show: the LED on PA5 off while B1 is held, blinking again after release, and `IDR13` changing in XPERIPHERALS.

Checkpoint C - Count the presses

Inside `USER CODE BEGIN 2` (before the loop):

```
int press_count = 0;
GPIO_PinState previous = GPIO_PIN_SET;   /* released */
```

Replace the code in `USER CODE BEGIN 3`:

```
GPIO_PinState current = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);

/* a press starts: released before, pressed now */
if (previous == GPIO_PIN_SET && current == GPIO_PIN_RESET) {
    press_count = press_count + 1;
    HAL_Delay(20);        /* ignore contact bounce */
}
previous = current;
```

Why `previous` and `current`? (click)

A press is a **change** (released → pressed), not a state. One reading shows only the state now, so the loop keeps the reading from the last pass in `previous` and compares it with `current` (1 = `GPIO_PIN_SET`, 0 = `GPIO_PIN_RESET`).

Loop pass	1	2	3	4	5	6	7
B1 (up = released)	up	up	**down**	down	down	up	up
`current`	1	1	**0**	0	0	1	1
`previous`	1	1	**1**	0	0	0	1
count A: `current` 0	0	0	1	2	3	3	3
count B: 1 → 0	0	0	**1**	1	1	1	1

Count A grows on every pass while B1 is held. Count B grows once per press: only in pass 3 is `previous` 1 and `current` 0.

- `previous = current;` at the end of the loop moves the `current` row one pass to the right.
- `previous` starts as `GPIO_PIN_SET` because B1 is released when the program starts.
- A real button bounces (1 0 1 0 0 …) for a few ms. The 20 ms delay after the first change skips it.

Below it, use `press_count % 4` to choose what the LED on PA5 does:

Presses so far	`press_count % 4`	LED on PA5
0, 4, 8, …	0	off
1, 5, …	1	on
2, 6, …	2	blinks slowly (toggle every 200 ms)
3, 7, …	3	blinks fast (toggle every 50 ms)

Check it in two steps:

1. **One count per press.** Set a breakpoint on `press_count = press_count + 1;` and Continue (F5). Press and **hold** B1: the program stops once. Press Continue while you still hold B1: it does **not** stop again. Release, press again: it stops again, and `press_count` in VARIABLES is 1 higher.
2. **Count and LED together.** Remove the breakpoint and Continue. Press B1 slowly and compare the LED with the table. Then press **Pause**, click `main` in CALL STACK, and read `press_count` in VARIABLES. (Pause usually stops inside `HAL_Delay`, whose variables do not include `press_count`.)

Show: the hold test from step 1, then five slow presses from the start: the LED lit and `press_count` = 5.

Optional (not graded) - Bare-metal rematch

In a second project, do Checkpoint B **without HAL GPIO calls**: enable the clocks (`RCC->AHB1ENR` bit 0 for GPIOA, bit 2 for GPIOC), reuse your Lab 3 `led_init`, `led_on`, `led_off`, and read `GPIOC->IDR` bit 13. Leave PC13 with no internal pull.

Common problems

Show the table (click)

What you see	First check
B1 does nothing	PC13 is `GPIO_Input` in the `.ioc`, and you generated code again. `IDR13` changes when you press.
LED behavior is inverted	`GPIO_PIN_SET` = released, `GPIO_PIN_RESET` = pressed.
`press_count` climbs while B1 is held	You test the level (`current` only), not the change (`previous` and `current`).
`press_count` jumps by 2-3 per press	The 20 ms delay is missing from the press branch.
Some presses are not counted	A long `HAL_Delay` blocks reading. Keep blink delays short. (Lecture 6 fixes this with interrupts.)
My code disappeared after Generate Code	It was outside `USER CODE`. Paste it again inside the region.
`...FLASH.ld: syntax error`	Build with **Ctrl+Shift+B**; check that `.vscode` is in the project folder.
`...FLASH.ld: syntax error` on your own laptop	Replace `FLASH.ld` with the file from the Home Setup Guide (step 6). Repeat after every Generate Code.

End of the session - Save to H

> ! Show the steps (click)
>
> Close VS Code, run the **Save to H** desktop shortcut, and check that `H:\EE_Labs\EE5203_L04_<studentnumber>` exists before you log out.

Acceptance checklist

- ☐ `.ioc`: PA5 output, PC13 input with no pull; code only inside `USER CODE`.
- ☐ Checkpoint A: blink and the three register fields.
- ☐ Checkpoint B: B1 stops the blink; `IDR13` shown.
- ☐ Checkpoint C: four behaviors, one count per press.

Individual oral check

Be ready for **one** question **or** one live change, chosen by your lab assistant. For a live change, predict the result first, then make the change.

Questions:

- Which field did `GPIO_MODE_OUTPUT_PP` change for PA5, and to what?
- Why does PC13 use `GPIO_NOPULL` on this board?
- Why does the counter compare `previous` and `current` instead of testing `current` alone?
- What does a successful build **not** prove?

Live changes:

- Set PC13 to pull-up in CubeMX. What does `PUPDR13` show? Does the LED behave differently?
- In Checkpoint B, make the LED on PA5 blink only while B1 is **pressed**.
- Remove the 20 ms delay. Press B1 a few times and watch `press_count`.

Submit

Submit `main.c`, the `.ioc` file, and one XPERIPHERALS screenshot showing `MODER5`, `MODER13` and `PUPDR13`.

Marks (10): pre-lab quiz on CATS 1 · A project, blink and register proof 3 · B button 2 · C press counter 2 · oral check (one question or one live change) 2. The bare-metal rematch is optional and not graded.